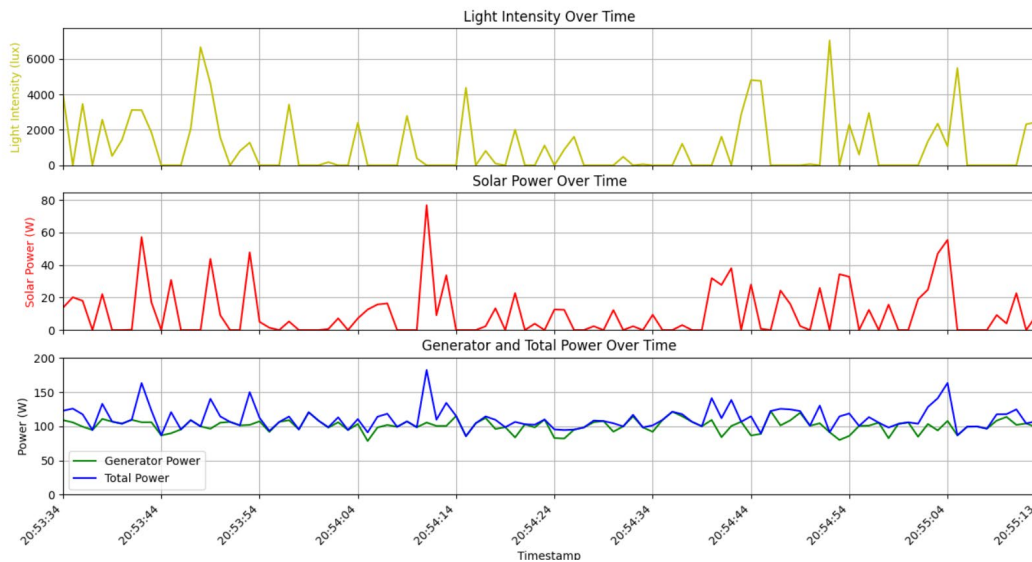


```
C:\Windows\System32\cmd.exe x + v
total_power_watt": 114.67999999999999, "light_intensity_lux": 4813.85, "generator_status": "ON",
{"timestamp": "2025-05-03 20:54:45", "solar_panel_power_watt": 0.0, "generator_power_watt": 88.92, "total_power_watt": 88.92, "light_intensity_lux": 4778.92, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:46", "solar_panel_power_watt": 0.0, "generator_power_watt": 122.41, "total_power_watt": 122.41, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:47", "solar_panel_power_watt": 20.33, "generator_power_watt": 101.35, "total_power_watt": 125.67999999999999, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:48", "solar_panel_power_watt": 15.94, "generator_power_watt": 108.81, "total_power_watt": 124.75, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:49", "solar_panel_power_watt": 2.47, "generator_power_watt": 119.86, "total_power_watt": 122.33, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:50", "solar_panel_power_watt": 0.0, "generator_power_watt": 100.91, "total_power_watt": 100.91, "light_intensity_lux": 67.03, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:51", "solar_panel_power_watt": 25.89, "generator_power_watt": 104.48, "total_power_watt": 130.37, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:52", "solar_panel_power_watt": 0.0, "generator_power_watt": 91.43, "total_power_watt": 91.43, "light_intensity_lux": 7052.7, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:53", "solar_panel_power_watt": 34.38, "generator_power_watt": 80.13, "total_power_watt": 114.50999999999999, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:54", "solar_panel_power_watt": 32.76, "generator_power_watt": 86.04, "total_power_watt": 118.80000000000001, "light_intensity_lux": 2304.51, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:55", "solar_panel_power_watt": 0.0, "generator_power_watt": 100.62, "total_power_watt": 100.62, "light_intensity_lux": 601.6, "generator_status": "ON"}

C:\Windows\System32\cmd.exe x + v
304|2025-05-03 20:52:12|0.0|100.88|100.88|2025-05-03 13:52:12
305|2025-05-03 20:52:13|126.88|105.21|132.10|2025-05-03 13:52:13
306|2025-05-03 20:52:14|18.18|103.36|121.54|0.0|2025-05-03 13:52:14
307|2025-05-03 20:52:15|11.19|115.87|127.06|3352.04|2025-05-03 13:52:15
308|2025-05-03 20:52:16|7.01|105.04|112.05|1035.28|2025-05-03 13:52:16
309|2025-05-03 20:52:17|0.0|108.04|108.04|0.0|2025-05-03 13:52:17
310|2025-05-03 20:52:18|19.59|97.53|117.12|1947.08|2025-05-03 13:52:18
311|2025-05-03 20:52:19|0.0|108.28|108.28|0.0|2025-05-03 13:52:19
312|2025-05-03 20:52:20|12.41|91.27|103.68|0.0|2025-05-03 13:52:20
313|2025-05-03 20:52:21|14.23|114.25|128.48|5079.63|2025-05-03 13:52:21
314|2025-05-03 20:52:22|126.18|90.71|116.89|0.0|2025-05-03 13:52:22
315|2025-05-03 20:52:23|0.0|84.81|84.81|1832.81|2025-05-03 13:52:23
316|2025-05-03 20:52:24|0.0|92.94|92.94|0.0|2025-05-03 13:52:24
317|2025-05-03 20:52:25|0.0|97.91|97.91|0.0|2025-05-03 13:52:25
318|2025-05-03 20:52:26|0.0|85.25|85.25|3539.70|2025-05-03 13:52:26
319|2025-05-03 20:52:27|5.15|86.87|92.02|0.0|2025-05-03 13:52:27
320|2025-05-03 20:52:28|0.0|97.12|97.12|3545.68|2025-05-03 13:52:28
321|2025-05-03 20:52:29|8.07|104.82|112.89|0.0|2025-05-03 13:52:29
322|2025-05-03 20:52:30|68.05|97.162|421731.1|2025-05-03 13:52:30
323|2025-05-03 20:52:31|0.0|111.07|111.07|3705.37|2025-05-03 13:52:31
324|2025-05-03 20:52:32|9.78|96.37|106.15|2651.53|2025-05-03 13:52:32
325|2025-05-03 20:52:33|0.0|101.82|101.82|224.17|2025-05-03 13:52:33
326|2025-05-03 20:52:34|0.0|96.23|96.23|2274.14|2025-05-03 13:52:34
sql>

Administrator Command Prompt - mosquitto_sub -h localhost -p 8883 -t "solar/sensor_value" -calle cert/cacert-cert cert/clientcert -key cert/
{"timestamp": "2025-05-03 20:54:42", "solar_panel_power_watt": 38.06, "generator_power_watt": 100.51, "total_power_watt": 138.57, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:43", "solar_panel_power_watt": 0.0, "generator_power_watt": 106.72, "total_power_watt": 106.72, "light_intensity_lux": 207.41, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:44", "solar_panel_power_watt": 20.02, "generator_power_watt": 86.66, "total_power_watt": 106.68, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:45", "solar_panel_power_watt": 0.0, "generator_power_watt": 88.92, "total_power_watt": 88.92, "light_intensity_lux": 4778.92, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:46", "solar_panel_power_watt": 0.0, "generator_power_watt": 122.41, "total_power_watt": 122.41, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:47", "solar_panel_power_watt": 24.33, "generator_power_watt": 101.35, "total_power_watt": 125.67999999999999, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:48", "solar_panel_power_watt": 15.94, "generator_power_watt": 108.81, "total_power_watt": 124.75, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:49", "solar_panel_power_watt": 2.47, "generator_power_watt": 119.86, "total_power_watt": 122.33, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:50", "solar_panel_power_watt": 0.0, "generator_power_watt": 100.91, "total_power_watt": 100.91, "light_intensity_lux": 67.03, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:51", "solar_panel_power_watt": 25.89, "generator_power_watt": 104.48, "total_power_watt": 130.37, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:52", "solar_panel_power_watt": 0.0, "generator_power_watt": 91.43, "total_power_watt": 91.43, "light_intensity_lux": 7052.7, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:53", "solar_panel_power_watt": 34.38, "generator_power_watt": 80.13, "total_power_watt": 114.50999999999999, "light_intensity_lux": 0.0, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:54", "solar_panel_power_watt": 32.76, "generator_power_watt": 86.04, "total_power_watt": 118.80000000000001, "light_intensity_lux": 2304.51, "generator_status": "ON"},
{"timestamp": "2025-05-03 20:54:55", "solar_panel_power_watt": 0.0, "generator_power_watt": 100.62, "total_power_watt": 100.62, "light_intensity_lux": 601.6, "generator_status": "ON"}

Solar Power Monitoring Dashboard
Generator Status: ON [Stop Generator]
```



power_intensity_datagen.py

```

import numpy as np
import time
import datetime
import json
import paho.mqtt.client as mqtt
from paho.mqtt.enums import CallbackAPIVersion
import os
import sys

# Status variable for generator
generator_on = False

# Callback when the client connects to the broker
def on_connect(client, userdata, flags, reason_code, properties=None):
    print(f'Connected with result code {reason_code}')
    # Subscribe to sensor data topic
    client.subscribe("sensor/sensor_value")
    # Subscribe to control topic for generator
    client.subscribe("control/generator")

# Callback when a message is received from the broker
def on_message(client, userdata, msg):
    global generator_on
    if msg.topic == "control/generator":
        command = msg.payload.decode()
        if command == "ON":
            generator_on = True
            print("Generator turned ON")
        elif command == "OFF":
            generator_on = False
            print("Generator turned OFF")

# Callback when a message is published
def on_publish(client, userdata, mid, reason_code=0, properties=None):
    pass

# Function to simulate solar panel power and light intensity based on current time
def calculate_solar_and_light(current_time):
    hour = current_time.hour
    minute = current_time.minute

    # Calculate time-based factor (peak at noon)
    time_factor = np.cos((hour - 12 + minute / 60) * np.pi / 12) ** 2

    # Set power and light to zero during nighttime (6 PM to 6 AM)
    if hour < 6 or hour >= 18:
        time_factor = 0

```

```

# Base values for solar power and light intensity
base_solar_power = 800 * time_factor # max 800W
base_light_intensity = 100000 * time_factor # max 100,000 lux

# Add random fluctuation
solar_power = base_solar_power + np.random.normal(0, 30)
light_intensity = base_light_intensity + np.random.normal(0, 3000)

# Ensure no negative values
solar_power = max(0, solar_power)
light_intensity = max(0, light_intensity)

return round(solar_power, 2), round(light_intensity, 2)

# Function to simulate generator power output
def calculate_generator_power():
    if generator_on:
        return round(100 + np.random.normal(0, 10), 2) # approx. 1200W ± 50W
    else:
        return 0.0

if __name__ == '__main__':
    # Define the path to certificate files
    parent_dir = r"D:\OneDrive - Digital Government Development Agency (Public Organization)\For
me\DADS\IoT\Code\week14"

    ca_file = os.path.join(parent_dir, "ca.crt")
    cert_file = os.path.join(parent_dir, "client.crt")
    key_file = os.path.join(parent_dir, "client.key")

    # Create MQTT client
    client = mqtt.Client(callback_api_version=CallbackAPIVersion.VERSION2)
    client.on_connect = on_connect
    client.on_message = on_message
    client.on_publish = on_publish

    # Set username and password for broker
    client.username_pw_set('pk', 'iot1234')

    # Set TLS for secure connection
    client.tls_set(
        ca_certs=ca_file,
        certfile=cert_file,
        keyfile=key_file
    )

    # Connect to MQTT broker
    client.connect("localhost", 8883, 60)
    client.loop_start()

```

```
# Infinite loop to send data every second
while True:
    now = datetime.datetime.now()
    date_time_str = now.strftime("%Y-%m-%d %H:%M:%S")

    # Calculate solar power and light intensity
    solar_power, light_intensity = calculate_solar_and_light(now)

    # Calculate generator power
    generator_power = calculate_generator_power()

    # Calculate total power
    total_power = solar_power + generator_power

    # Create data payload
    data = {
        'timestamp': date_time_str,
        'solar_panel_power_watt': solar_power,
        'generator_power_watt': generator_power,
        'total_power_watt': total_power,
        'light_intensity_lux': light_intensity,
        'generator_status': "ON" if generator_on else "OFF"
    }

    # Convert dictionary to JSON string
    data_json_str = json.dumps(data)

    # Print to console
    print(data_json_str)

    # Publish data to MQTT topic
    client.publish("sensor/sensor_value", data_json_str)

    # Wait for 1 second before sending the next message
    time.sleep(1)
```

power_intensity_plotter.py

```

import paho.mqtt.client as mqtt
from paho.mqtt.enums import CallbackAPIVersion
import json
import matplotlib.pyplot as plt
from collections import deque
import threading
import os
import tkinter as tk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import time

# =====
# Configuration Section
# =====

# Maximum number of data points to display
max_len = 100

# Deques for storing time-series data
timestamps = deque(maxlen=max_len)
solar_powers = deque(maxlen=max_len)
generator_powers = deque(maxlen=max_len)
total_powers = deque(maxlen=max_len)
light_intensities = deque(maxlen=max_len)

# Global variables
generator_status = "OFF"
client = None

# =====
# MQTT Callback Functions
# =====

def on_connect(client, userdata, flags, reason_code, properties=None):
    """Called when the client connects to the MQTT broker."""
    print(f"Connected with result code {reason_code}")
    client.subscribe("sensor/sensor_value")

def on_message(client, userdata, msg):
    """Called when a new MQTT message is received."""
    global generator_status
    data = json.loads(msg.payload.decode())
    print(data)

    # Append received data to respective deques
    timestamps.append(data['timestamp'])
    solar_powers.append(data['solar_panel_power_watt'])

```

```

generator_powers.append(data['generator_power_watt'])
total_powers.append(data['total_power_watt'])
light_intensities.append(data['light_intensity_lux'])
generator_status = data.get('generator_status', generator_status)

# Update GUI status label and buttons
if 'status_label' in userdata and userdata['status_label']:
    userdata['status_label'].config(text=f"Generator Status: {generator_status}")
    if generator_status == "ON":
        userdata['on_button'].config(state=tk.DISABLED, bg="gray")
        userdata['off_button'].config(state=tk.NORMAL, bg="red")
    else:
        userdata['on_button'].config(state=tk.NORMAL, bg="green")
        userdata['off_button'].config(state=tk.DISABLED, bg="gray")

# =====
# Button Handlers
# =====

def turn_on_generator():
    """Publish message to turn ON the generator."""
    if client:
        client.publish("control/generator", "ON")
        print("Sending generator ON command")

def turn_off_generator():
    """Publish message to turn OFF the generator."""
    if client:
        client.publish("control/generator", "OFF")
        print("Sending generator OFF command")

# =====
# Dashboard (GUI)
# =====

def create_dashboard():
    """Create the main dashboard UI with plots and controls."""
    root = tk.Tk()
    root.title("Solar Power Monitoring Dashboard")
    root.geometry("1200x800")

    # Control section
    control_frame = tk.Frame(root)
    control_frame.pack(pady=10)

    status_label = tk.Label(control_frame, text=f"Generator Status: {generator_status}", font=("Arial",
14))
    status_label.pack(side=tk.LEFT, padx=20)

    on_button = tk.Button(control_frame, text="Start Generator", command=turn_on_generator,
        bg="green", fg="white", font=("Arial", 12), padx=10)

```

```

on_button.pack(side=tk.LEFT, padx=10)

off_button = tk.Button(control_frame, text="Stop Generator", command=turn_off_generator,
                        bg="gray", fg="white", font=("Arial", 12), padx=10, state=tk.DISABLED)
off_button.pack(side=tk.LEFT, padx=10)

# Plot section using matplotlib
fig = plt.Figure(figsize=(12, 8), dpi=100)

# Light intensity plot
ax1 = fig.add_subplot(3, 1, 1)
line1, = ax1.plot([], [], 'y-', label='Light Intensity')
ax1.set_ylabel('Light Intensity (lux)', color='y')
ax1.set_title('Light Intensity Over Time')
ax1.grid(True)

# Solar power plot
ax2 = fig.add_subplot(3, 1, 2)
line2, = ax2.plot([], [], 'r-', label='Solar Power')
ax2.set_ylabel('Solar Power (W)', color='r')
ax2.set_title('Solar Power Over Time')
ax2.grid(True)

# Generator and total power plot
ax3 = fig.add_subplot(3, 1, 3)
line3, = ax3.plot([], [], 'g-', label='Generator Power')
line4, = ax3.plot([], [], 'b-', label='Total Power')
ax3.set_ylabel('Power (W)')
ax3.set_xlabel('Timestamp')
ax3.set_title('Generator and Total Power Over Time')
ax3.legend()
ax3.grid(True)

# Embed plot in tkinter window
canvas = FigureCanvasTkAgg(fig, master=root)
canvas.draw()
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

# Plot update function (called every second)
def update_plot():
    if timestamps:
        x_range = range(len(timestamps))

        # Update all lines with new data
        line1.set_data(x_range, list(light_intensities))
        line2.set_data(x_range, list(solar_powers))
        line3.set_data(x_range, list(generator_powers))
        line4.set_data(x_range, list(total_powers))

        # Update axis limits
        for ax in [ax1, ax2, ax3]:

```

```

        ax.set_xlim(0, len(timestamps) - 1)

    if light_intensities:
        ax1.set_ylim(0, max(light_intensities) * 1.1)
    if solar_powers:
        ax2.set_ylim(0, max(solar_powers) * 1.1)
    if total_powers:
        max_power = max(max(total_powers), max(generator_powers) if generator_powers else 0)
        ax3.set_ylim(0, max_power * 1.1)

    # Update X-tick labels
    if len(timestamps) > 10:
        tick_indices = [i for i in range(0, len(timestamps), len(timestamps)//10)]
        if len(timestamps)-1 not in tick_indices:
            tick_indices.append(len(timestamps)-1)

        for ax in [ax1, ax2, ax3]:
            ax.set_xticks(tick_indices)
            if ax == ax3:
                ax.set_xticklabels([timestamps[i].split()[1] for i in tick_indices],
                                    rotation=45, ha='right')
            else:
                ax.set_xticklabels([])

    canvas.draw()

    # Schedule the next update
    root.after(1000, update_plot)

update_plot()

# Return root window and UI components
return root, status_label, on_button, off_button

# =====
# Main Program
# =====

if __name__ == "__main__":
    # Path to TLS certificate files
    parent_dir = r"D:\OneDrive - Digital Government Development Agency (Public Organization)\For me\DADS\IoT\Code\week14"
    ca_file = os.path.join(parent_dir, "ca.crt")
    cert_file = os.path.join(parent_dir, "client.crt")
    key_file = os.path.join(parent_dir, "client.key")

    # Start the dashboard
    root, status_label, on_button, off_button = create_dashboard()

    # Initialize MQTT client
    client = mqtt.Client(callback_api_version=CallbackAPIVersion.VERSION2)

```

```
client.on_connect = on_connect
client.on_message = on_message

# Share tkinter widgets with MQTT callback
client.user_data_set({
    'status_label': status_label,
    'on_button': on_button,
    'off_button': off_button
})

# Set MQTT credentials
client.username_pw_set('pk', 'iot1234')

# Enable TLS security
client.tls_set(
    ca_certs=ca_file,
    certfile=cert_file,
    keyfile=key_file
)

# Connect to broker
client.connect("localhost", 8883, 60)

# Start MQTT processing in a background thread
mqtt_thread = threading.Thread(target=client.loop_forever, daemon=True)
mqtt_thread.start()

# Start GUI main loop
try:
    root.mainloop()
except KeyboardInterrupt:
    print("Exiting...")
finally:
    client.disconnect()
```

power_intensity_storage.py

```

import paho.mqtt.client as mqtt
from paho.mqtt.enums import CallbackAPIVersion
import sqlite3
import json
import os
import datetime

# MQTT Configuration
MQTT_HOST = "localhost"
MQTT_PORT = 8883
MQTT_USER = "pk"
MQTT_PASSWORD = "iot1234"
MQTT_TOPIC = "sensor/sensor_value"

# Define the SQLite database file path (same directory as this script)
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DB_FILE = os.path.join(SCRIPT_DIR, "solar_power_data.db")

# Called when the client successfully connects to the MQTT broker
def on_connect(client, userdata, flags, reason_code, properties=None):
    print(f"Connected to MQTT Broker (code: {reason_code})")
    client.subscribe(MQTT_TOPIC)
    print(f"Subscribed to topic: {MQTT_TOPIC}")

# Called whenever a message is received from the subscribed topic
def on_message(client, userdata, msg):
    try:
        # Decode the JSON payload
        payload = msg.payload.decode('utf-8')
        data = json.loads(payload)

        # Extract data from the JSON
        timestamp = data.get('timestamp')
        solar_power = data.get('solar_panel_power_watt')
        generator_power = data.get('generator_power_watt')
        total_power = data.get('total_power_watt')
        light_intensity = data.get('light_intensity_lux')
        generator_status = data.get('generator_status')

        # Insert data into SQLite database
        db_conn = userdata['db_conn']
        cursor = db_conn.cursor()

        # SQL command to insert a new record
        sql = """
        INSERT INTO solar_power_data
        (timestamp, solar_power, generator_power, total_power, light_intensity, generator_status)
    
```

```

VALUES (?, ?, ?, ?, ?, ?)
"""

cursor.execute(sql, (
    timestamp,
    solar_power,
    generator_power,
    total_power,
    light_intensity,
    generator_status
))

db_conn.commit()
print(f'Data inserted successfully: {timestamp}')

except Exception as e:
    print(f'Error occurred: {e}')

# Initializes the SQLite database and creates the table if it does not exist
def setup_database():
    print(f'Database location: {DB_FILE}')

    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    # Create table if it doesn't exist
    cursor.execute("""
CREATE TABLE IF NOT EXISTS solar_power_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT,
    solar_power REAL,
    generator_power REAL,
    total_power REAL,
    light_intensity REAL,
    generator_status TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

    conn.commit()
    print("Database table initialized.")

    return conn

# Main entry point of the script
def main():
    # Prepare the database
    db_conn = setup_database()

    # Define the path for the TLS certificate files

```

```

parent_dir = r"D:\OneDrive - Digital Government Development Agency (Public Organization)\For
me\DADS\IoT\Code\week14"
ca_file = os.path.join(parent_dir, "ca.crt")
cert_file = os.path.join(parent_dir, "client.crt")
key_file = os.path.join(parent_dir, "client.key")

# Create MQTT client with appropriate callback API version
client = mqtt.Client(callback_api_version=CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

# Set user data (used in message callback for DB access)
client.user_data_set({'db_conn': db_conn})

# Set username and password for MQTT authentication
client.username_pw_set(MQTT_USER, MQTT_PASSWORD)

# Configure TLS/SSL connection using certificates
client.tls_set(
    ca_certs=ca_file,
    certfile=cert_file,
    keyfile=key_file
)

try:
    # Connect to the MQTT broker
    print(f"Connecting to MQTT Broker at {MQTT_HOST}:{MQTT_PORT}...")
    client.connect(MQTT_HOST, MQTT_PORT, 60)

    # Start listening for messages (blocking loop)
    print("Listening for incoming messages... (Press Ctrl+C to stop)")
    client.loop_forever()

except KeyboardInterrupt:
    print("\nStopped by user.")
except Exception as e:
    print(f"Unexpected error: {e}")
finally:
    # Disconnect MQTT and close database connection
    print("Shutting down...")
    client.disconnect()
    db_conn.close()
    print("Disconnected and closed database.")

if __name__ == "__main__":
    main()

```